CRANE-midterm

Tanner Crane

October 2026

1

Expand $f(x+h)$ and $f(x+2h)$ about x using Taylor's Theorem:

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2}f''(x) + \frac{h^3}{6}f'''(x) + \mathcal{O}(h^4)$$

$$f(x+2h) = f(x) + 2hf'(x) + 2h^2f''(x) + \frac{4h^3}{3}f'''(x) + \mathcal{O}(h^4)$$

Substitute both expansions into the linear combination $-3f(x) + 4f(x+h) - f(x+2h)$:

$$-3f(x) + 4f(x+h) - f(x+2h) =$$

$$(-3+4-1)f(x)+(4h-2h)f'(x)+(2h^2-2h^2)f''(x)+\left(\frac{4}{6}-\frac{4}{3}\right)h^3f'''(x)+\mathcal{O}(h^4)$$

$$= 2hf'(x) - \frac{2}{3}h^3f'''(x) + \mathcal{O}(h^4)$$

Divide by $2h$ and solve for $f'(x)$:

$$f'(x) = \frac{1}{2h}\left(-3f(x) + 4f(x+h) - f(x+2h)\right) + \frac{h^2}{3}f'''(\xi)$$

for some $\xi \in (x, x+2h)$. Thus, the error term is $\frac{h^2}{3}f'''(\xi)$ (or $\mathcal{O}(h^2)$ using asymptotic notation). Assuming the error is Exact - Approx.

2

Let α be the exact root such that $f(\alpha) = 0$, and define the error at step n as $e_n = x_n - \alpha$. Subtracting α from both sides of the iteration $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_0)}$ yields:

$$e_{n+1} = e_n - \frac{f(x_n)}{f'(x_0)}$$

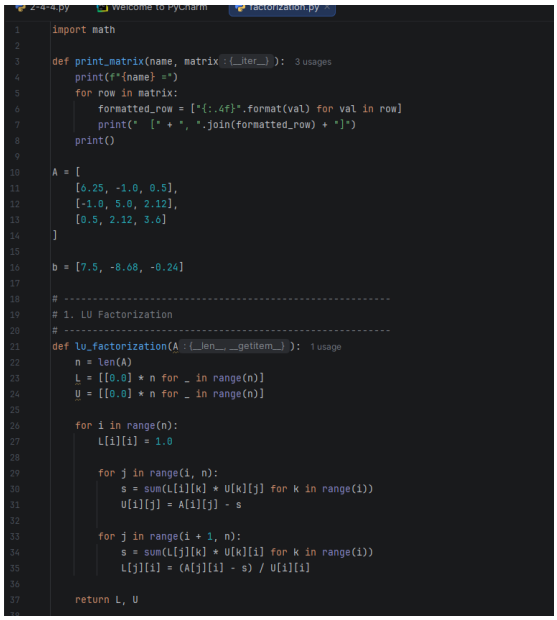
Expanding $f(x_n)$ around α using a Taylor series gives $f(x_n) = f(\alpha) + f'(\alpha)e_n + O(e_n^2) = f'(\alpha)e_n + O(e_n^2)$. Substituting this into the error equation:

$$e_{n+1} = e_n - \frac{f'(\alpha)e_n + O(e_n^2)}{f'(x_0)} = \left(1 - \frac{f'(\alpha)}{f'(x_0)}\right) e_n + O(e_n^2)$$

Taking the dominant leading-order term as $x_n \to \alpha$ gives $e_{n+1} \approx Ce_n^s$, with $s = 1$ (linear convergence)
$C = 1 - \frac{f'(\alpha)}{f'(x_0)}$ (or $C = 1 - \frac{f'(\eta_n)}{f'(x_0)}$ for some η_n between x_n and α if written as an exact equality via the Mean Value Theorem).

3

The code below uses Python with nested loops rather than standard libraries for math such as numpy or scipy.



```python
import math

def print_matrix(name, matrix):  3 usages
    print(f"{name} =")
    for row in matrix:
        formatted_row = ["{:.4f}".format(val) for val in row]
        print("  [" + ", ".join(formatted_row) + "]")
    print()

A = [
    [6.25, -1.0, 0.5],
    [-1.0, 5.0, 2.12],
    [0.5, 2.12, 3.6]
]

b = [7.5, -8.68, -0.24]

# ----------------------------------------------------------
# 1. LU Factorization
# ----------------------------------------------------------
def lu_factorization(A):  1 usage
    n = len(A)
    L = [[0.0] * n for _ in range(n)]
    U = [[0.0] * n for _ in range(n)]

    for i in range(n):
        L[i][i] = 1.0

        for j in range(i, n):
            s = sum(L[i][k] * U[k][j] for k in range(i))
            U[i][j] = A[i][j] - s

        for j in range(i + 1, n):
            s = sum(L[j][k] * U[k][i] for k in range(i))
            L[j][i] = (A[j][i] - s) / U[i][i]

    return L, U
```

Figure 1: LU Factorization



```python
# ----------------------------------------------------------
# 2. Cholesky Factorization (A = LL^T)
# ----------------------------------------------------------
def cholesky_factorization(A):  1 usage
    n = len(A)
    L = [[0.0] * n for _ in range(n)]

    for i in range(n):
        for j in range(i + 1):
            s = sum(L[i][k] * L[j][k] for k in range(j))

            if i == j:
                L[i][j] = math.sqrt(A[i][i] - s)
            else:
                L[i][j] = (A[i][j] - s) / L[j][j]

    return L


L_lu, U_lu = lu_factorization(A)
print("--- LU Factorization Output ---")
print_matrix( name: "L", L_lu)
print_matrix( name: "U", U_lu)

L_chol = cholesky_factorization(A)
print("--- Cholesky Factorization Output ---")
print_matrix( name: "L", L_chol)
```

Figure 2: Cholesky Algorithm



```
C:\Users\Tanner\PycharmProjects\WelcomeScreen\.venv\Scripts\python.exe C:\Users\Tanner\PycharmProjects\WelcomeScreen\factorization.py
--- LU Factorization Output ---
L =
  [1.0000, 0.0000, 0.0000]
  [-0.1600, 1.0000, 0.0000]
  [0.0800, 0.4545, 1.0000]

U =
  [6.2500, -1.0000, 0.5000]
  [0.0000, 4.8400, 2.2000]
  [0.0000, 0.0000, 2.5600]

--- Cholesky Factorization Output ---
L =
  [2.5000, 0.0000, 0.0000]
  [-0.4000, 2.2000, 0.0000]
  [0.2000, 1.0000, 1.6000]


Process finished with exit code 0
```

Figure 3: Output

4

$$x - \frac{1}{2}\left(\cos(x/2) - \left|x - \frac{1}{2}\right|\right) = 0$$

$$x = \frac{1}{2}\left(\cos(x/2) - \left|x - \frac{1}{2}\right|\right)$$

Let $T(x) = \frac{1}{2}\left(\cos(x/2) - \left|x - \frac{1}{2}\right|\right)$. Finding a root of $f(x) = 0$ is equivalent to finding a fixed point $x = T(x)$

Let $I = [0.45, 0.55]$. The goal is to show for any $x \in I$, the output T(x) is also in I. Bounding the absolute value term:

$$0.45 \leq x \leq 0.55 \implies -0.05 \leq x - 0.5 \leq 0.05$$

$$0 \leq \left|x - \frac{1}{2}\right| \leq 0.05$$

Cosine term for $x \in [0.45, 0.55]$, the argument $x/2 \in [0.225, 0.275]$:

$$\cos(0.275) \leq \cos(x/2) \leq \cos(0.225)$$

$$0.9624 \lesssim \cos(x/2) \lesssim 0.9748$$

Now we find the minimum and maximum values for T(x) by analyzing the upper and lower bounds, respectively. (maximizing the positive term and minimizing the subtracted term, and vice versa):

$$T(x) \leq \frac{1}{2}(\max\cos(x/2) - \min|x - 0.5|) \leq \frac{1}{2}(0.9748 - 0) = 0.4874$$

$$T(x) \geq \frac{1}{2}(\min\cos(x/2) - \max|x - 0.5|) \geq \frac{1}{2}(0.9624 - 0.05) = 0.4562$$

Since $0.45 \leq 0.4562 \leq T(x) \leq 0.4874 \leq 0.55$, it follows that $T(x) \in [0.45, 0.55]$ and is thus self-mapping. □

To prove T(x) is contractive in I, We must show there exists a Lipschitz constant $0 \leq L < 1$ such that $|T(x) - T(y)| \leq L|x - y|$ for all $x, y \in I$.

Evaluate the derivative $T'(x)$ piecewise for $x \neq 0.5$ because of the absolute value at $x = 0.5$:
For $x \in [0.45, 0.5)$: $T(x) = \frac{1}{2}\cos(x/2) + \frac{1}{2}(x - 0.5) \implies T'(x) = -\frac{1}{4}\sin(x/2) + \frac{1}{2}$
For $x \in (0.5, 0.55]$: $T(x) = \frac{1}{2}\cos(x/2) - \frac{1}{2}(x - 0.5) \implies T'(x) = -\frac{1}{4}\sin(x/2) - \frac{1}{2}$

For $x \in [0.45, 0.55]$, $\sin(x/2)$ is positive and increasing, bounded by $\sin(0.275) \approx 0.2715$.

For $x < 0.5$: $|T'(x)| = \left|\frac{1}{2} - \frac{1}{4}\sin(x/2)\right| \leq 0.5$.
For $x > 0.5$: $|T'(x)| = \left|-\frac{1}{2} - \frac{1}{4}\sin(x/2)\right| = \frac{1}{2} + \frac{1}{4}\sin(x/2) \leq 0.5 + \frac{1}{4}(0.2715) \approx 0.5679$.

The function is continuous everywhere on I and differentiable everywhere except $x = 0.5$. By the Mean Value Theorem, the maximum slope dictates the Lipschitz constant. Since $\sup|T'(x)| \approx 0.5679 \leq L < 1$, the function $T(x)$ is strictly contractive on this interval
In conclusion, the Contraction Mapping Theorem, because $T(x)$ is a contraction mapping on the closed interval $[0.45, 0.55]$ and maps this interval into itself, says there exists exactly one fixed point x^* in $[0.45, 0.55]$ such that $T(x^*) = x^*$. Consequently, there is a unique solution to the equation $f(x) = 0$ on this interval.

2.

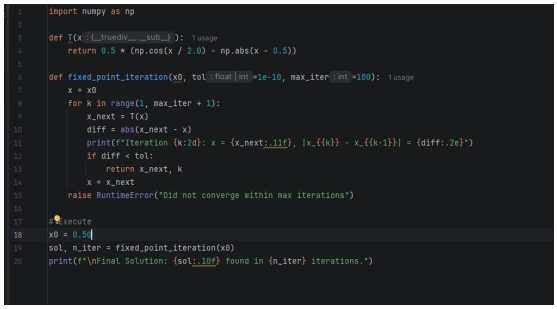


Figure 4: Python function for Fixed Point Iterations

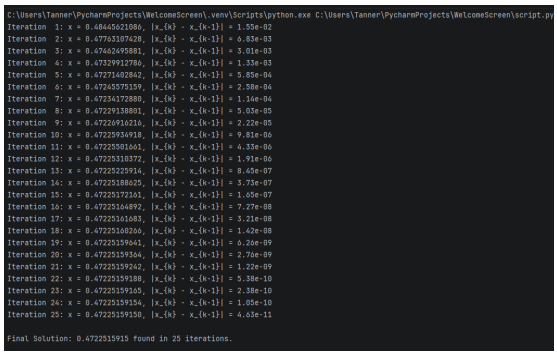


Figure 5: Output from Console

Because Part 1 established that $T(x)$ is a contraction mapping on $[0.45, 0.55]$, any choice of $x_0 \in [0.45, 0.55]$ is guaranteed by the Contractive Mapping Theorem to converge to the unique fixed point. Setting $x_0 = 0.50$ (the midpoint) is standard, minimizes the maximum initial distance to any point in the interval to 0.05, and simplifies the first evaluation since $|0.5 - 0.5| = 0$. So choice of initial guess x_0 should be 0.5.

3. Assuming target tolerance: $\epsilon = 10^{-10}$.
A Priori estimate:

$$|x_n - x^*| \leq \frac{L^n}{1-L}|x_1 - x_0| \leq \epsilon$$

$$\frac{0.5^n}{1-0.5}(0.0155324) \leq 10^{-10} \implies 0.5^n(0.0310648) \leq 10^{-10}$$

$$0.5^n \leq 3.219 \times 10^{-9} \implies n \geq \frac{\ln(3.219 \times 10^{-9})}{\ln(0.5)} \approx \frac{-19.5539}{-0.69315} \approx 28.21$$

Rounding up gives $n = 29$ iterations.

A Posteriori Estimate:

$$|x_n - x^*| \leq \frac{L}{1-L}|x_n - x_{n-1}|$$

For $L = 0.5$, $\frac{L}{1-L} = \frac{0.5}{0.5} = 1$, which gives:

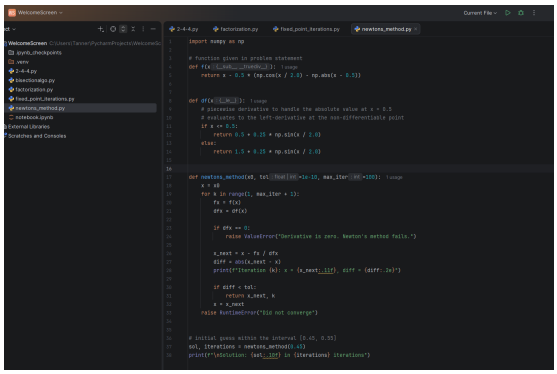
$$|x_n - x^*| \leq |x_n - x_{n-1}|$$

Monitoring $|x_n - x_{n-1}| \leq 10^{-10}$ in the iteration table yields:
At $n = 24$: $|x_{24} - x_{23}| \approx 3.9 \times 10^{-10}$
At $n = 25$: $|x_{25} - x_{24}| \approx 1.8 \times 10^{-10}$
At $n = 26$: $|x_{26} - x_{25}| \approx 8.6 \times 10^{-11} \leq 10^{-10}$
This gives $n = 26$ iterations.

Thus, the a posteriori estimate is more accurate. This is because the a posteriori estimate recalculates the bound using real-time information from the most recent iterate ($|x_n - x_{n-1}|$), reflecting the local contractive behavior directly at each stage and stopping much closer to the true convergence step.

4. Analysis: The difficulty arises from the absolute value term $|x - 0.5|$ in $f(x)$. The problem states that $f'(x)$ does not exist at certain points within the interval $[0.45, 0.55]$. Specifically, $f(x)$ is continuous everywhere but not differentiable at exactly $x = 0.5$. Because $0.5 \in [0.45, 0.55]$, standard Newton's method ($x_{k+1} = x_k - f(x_k)/f'(x_k)$) will fail if the initial guess x_0 is set to 0.5, or if any subsequent iteration lands exactly on this cusp, due to a division by an undefined derivative.

To resolve this problem two modifications are necessary. First, the derivative $f'(x)$ must be defined piecewise in the code, assigning a valid one-sided



```python
import numpy as np

# Function given in problem statement
def f(x):
    return x - 0.5 * (np.cos(x / 2.0) - np.abs(x - 0.5))

def df(x):
    # piecewise derivative to handle the absolute value at x = 0.5
    # evaluates to the left-derivative at the non-differentiable point
    if x <= 0.5:
        return 0.5 + 0.25 * np.sin(x / 2.0)
    else:
        return 1.5 + 0.25 * np.sin(x / 2.0)

def newtons_method(x0, tol=1e-10, max_iter=100):
    x = x0
    for k in range(1, max_iter + 1):
        fx = f(x)
        dfx = df(x)

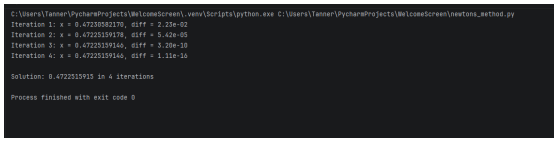
        if dfx == 0:
            raise ValueError("Derivative is zero. Newton's method fails.")

        x_next = x - fx / dfx
        diff = abs(x_next - x)
        print(f"Iteration {k}: x = {x_next:.11f}, diff = {diff:.2e}")

        if diff < tol:
            return x_next, k
        x = x_next
    raise RuntimeError("Did not converge")


# Initial guess within the interval [0.45, 0.55]
sol, iterations = newtons_method(0.45)
print(f"\nSolution: {sol:.10f} in {iterations} iterations")
```

Figure 6: Newton's Method Algorithm



```
C:\Users\Tanner\PycharmProjects\WelcomeScreen\.venv\Scripts\python.exe C:\Users\Tanner\PycharmProjects\WelcomeScreen\newtons_method.py
Iteration 1: x = 0.47230582170, diff = 2.23e-02
Iteration 2: x = 0.47225159178, diff = 5.42e-05
Iteration 3: x = 0.47225159144, diff = 3.20e-10
Iteration 4: x = 0.47225159146, diff = 1.11e-16

Solution: 0.4722515915 in 4 iterations

Process finished with exit code 0
```

Figure 7: Newton's Method Console Output

limit (such as the left derivative) exactly at $x = 0.5$ to prevent fatal errors. Second, because the actual root $x^* \approx 0.4716$ resides in the smooth subdomain where $x < 0.5$, setting the initial guess x_0 to a point strictly less than 0.5 (e.g., $x_0 = 0.45$) guarantees the algorithm stays in the continuously differentiable region containing the root.

5. Iterations and Convergence speed:
It requires 4 iterations to calculate the solution to within 10 decimal places using Newton's method as demonstrated in the above code. Newton's method demonstrates quadratic convergence near simple roots. As seen in the iteration data, once the guess is in the immediate vicinity of the root, the error is roughly squared at each step, so the number of correct decimal places practically doubles with every iteration.
The fixed-point iteration $x_{n+1} = T(x_n)$ evaluated previously only exhibits linear convergence. The error at each step decreases by a roughly constant multiplicative factor (the Lipschitz constant $L \approx 0.5$). Consequently, fixed-point iteration takes substantially more steps (approximately 25 iterations) to reach the same 10-decimal precision, making Newton's method significantly faster.

6. Define the residual function for this specific fixed-point iteration

$$(x_{n+1} = T(x_n))$$

as

$$R(x) = x - T(x)$$

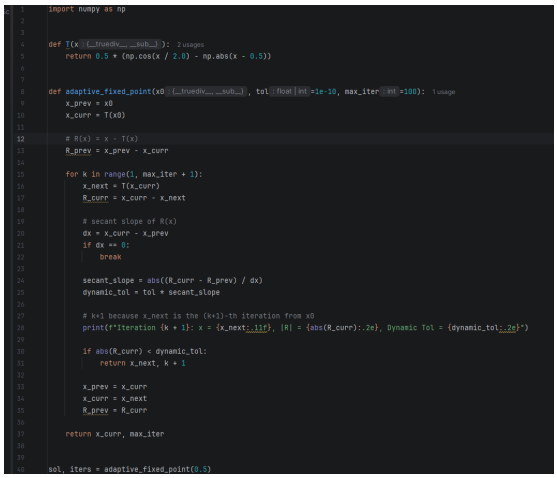
The absolute residual evaluated at step n is the difference between consecutive iterates: $|R(x_n)| = |x_n - x_{n+1}|$.

The strategy calculates a dynamic threshold τ_n at each step by multiplying the base tolerance ϵ by the local secant slope S_n over the last two evaluations:

$$\tau_n = \epsilon \cdot S_n = \epsilon \cdot \left| \frac{R(x_n) - R(x_{n-1})}{x_n - x_{n-1}} \right|$$

The iteration halts when $|R(x_n)| < \tau_n$

When executed on the midterm function with $\epsilon = 10^{-10}$, the adaptive strategy requires 26 iterations to halt.
Comparisons:
A Priori Estimate: 29 iterations (Static, overly conservative).
A Posteriori Estimate: 26 iterations (Responsive, utilizing the theoretical constant $L = 0.5$).
Adaptive Strategy: 26 iterations (Highly responsive, dynamically computing the local slope).



```python
import numpy as np


def T(x: (_truediv__, __sub__)): 2 usages
    return 0.5 + (np.cos(x / 2.0) - np.abs(x - 0.5))


def adaptive_fixed_point(x0: (_truediv__, __sub__), tol: float | int =1e-10, max_iter: int =100): 1 usage
    x_prev = x0
    x_curr = T(x0)

    # R(x) = x - T(x)
    R_prev = x_prev - x_curr

    for k in range(1, max_iter + 1):
        x_next = T(x_curr)
        R_curr = x_curr - x_next

        # secant slope of R(x)
        dx = x_curr - x_prev
        if dx == 0:
            break

        secant_slope = abs((R_curr - R_prev) / dx)
        dynamic_tol = tol * secant_slope

        # k+1 because x_next is the (k+1)-th iteration from x0
        print(f"Iteration {k + 1}: x = {x_next:.11f}, |R| = {abs(R_curr):.2e}, Dynamic Tol = {dynamic_tol:.2e}")

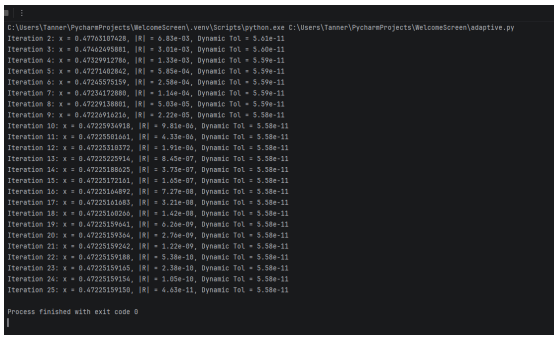
        if abs(R_curr) < dynamic_tol:
            return x_next, k + 1

        x_prev = x_curr
        x_curr = x_next
        R_prev = R_curr

    return x_curr, max_iter


sol, iters = adaptive_fixed_point(0.5)
```

Figure 8: Adaptive Algorithm



```
C:\Users\Tanner\PycharmProjects\WelcomeScreen\.venv\Scripts\python.exe C:\Users\Tanner\PycharmProjects\WelcomeScreen\adaptive.py
Iteration 2: x = 0.47763107428, |R| = 6.83e-03, Dynamic Tol = 3.61e-11
Iteration 3: x = 0.47662495881, |R| = 1.01e-03, Dynamic Tol = 5.60e-11
Iteration 4: x = 0.47329912786, |R| = 1.33e-03, Dynamic Tol = 5.59e-11
Iteration 5: x = 0.47271402842, |R| = 5.85e-04, Dynamic Tol = 5.59e-11
Iteration 6: x = 0.47265575159, |R| = 2.58e-04, Dynamic Tol = 5.59e-11
Iteration 7: x = 0.47234172888, |R| = 1.14e-04, Dynamic Tol = 5.59e-11
Iteration 8: x = 0.47229138881, |R| = 5.03e-05, Dynamic Tol = 5.59e-11
Iteration 9: x = 0.47226916216, |R| = 2.22e-05, Dynamic Tol = 5.58e-11
Iteration 10: x = 0.47225934918, |R| = 9.81e-06, Dynamic Tol = 5.58e-11
Iteration 11: x = 0.47225501661, |R| = 4.33e-06, Dynamic Tol = 5.58e-11
Iteration 12: x = 0.47225310372, |R| = 1.91e-06, Dynamic Tol = 5.58e-11
Iteration 13: x = 0.47225225914, |R| = 8.45e-07, Dynamic Tol = 5.58e-11
Iteration 14: x = 0.47225188025, |R| = 3.73e-07, Dynamic Tol = 5.58e-11
Iteration 15: x = 0.47225172161, |R| = 1.65e-07, Dynamic Tol = 5.58e-11
Iteration 16: x = 0.47225164092, |R| = 7.27e-08, Dynamic Tol = 5.58e-11
Iteration 17: x = 0.47225161483, |R| = 3.21e-08, Dynamic Tol = 5.58e-11
Iteration 18: x = 0.47225160266, |R| = 1.42e-08, Dynamic Tol = 5.58e-11
Iteration 19: x = 0.47225159641, |R| = 6.26e-09, Dynamic Tol = 5.58e-11
Iteration 20: x = 0.47225159364, |R| = 2.76e-09, Dynamic Tol = 5.58e-11
Iteration 21: x = 0.47225159242, |R| = 1.22e-09, Dynamic Tol = 5.58e-11
Iteration 22: x = 0.47225159188, |R| = 5.38e-10, Dynamic Tol = 5.58e-11
Iteration 23: x = 0.47225159165, |R| = 2.38e-10, Dynamic Tol = 5.58e-11
Iteration 24: x = 0.47225159154, |R| = 1.05e-10, Dynamic Tol = 5.58e-11
Iteration 25: x = 0.47225159150, |R| = 4.63e-11, Dynamic Tol = 5.58e-11

Process finished with exit code 0
```

Figure 9: Output

5

1. Let $x_2 = \frac{x_0+x_1}{2}$ be the midpoint of a bracketing interval ($f_0 f_1 < 0$), with step size $d = x_2 - x_0$. We define $g(x) = f(x)e^{ax}$ such that $(x_0, g(x_0))$, $(x_2, g(x_2))$, and $(x_1, g(x_1))$ are collinear.

Collinearity requires the midpoint $g(x_2)$ to be the exact average of $g(x_0)$ and $g(x_1)$:

$$f_2 e^{ax_2} = \frac{1}{2}(f_0 e^{ax_0} + f_1 e^{ax_1})$$

Multiplying by $2e^{-ax_2}$ and substituting $d = x_2 - x_0 = x_1 - x_2$ yields:

$$2f_2 = f_0 e^{-ad} + f_1 e^{ad}$$

Letting $u = e^{ad}$ and multiplying through by u, we obtain the quadratic:

$$f_1 u^2 - 2f_2 u + f_0 = 0$$

The roots are $u = \frac{f_2 \pm \sqrt{f_2^2 - f_0 f_1}}{f_1}$. Since $f_0 f_1 < 0$, the discriminant is strictly positive (ensuring real roots), and the product of the roots (f_0/f_1) is strictly negative (ensuring exactly one positive root and one negative root).Because $u = e^{ad}$ must be positive, u is uniquely selected by the single positive root, meaning $a = \frac{1}{d}\ln(u)$ is uniquely defined.

2. To guarantee $u > 0$, the sign chosen in the quadratic formula must match the sign of f_1. Letting $W = \sqrt{f_2^2 - f_0 f_1}$, the unique positive root is:

$$u = \frac{f_2 + \text{sgn}(f_1)W}{f_1}$$

Next, we apply the secant method (false position) to the line $g(x)$ passing through x_0 and x_2 to find its x-intercept, x_3:

$$x_3 = x_2 - \frac{g(x_2)(x_2 - x_0)}{g(x_2) - g(x_0)} = x_2 - d\frac{f_2 e^{ax_2}}{f_2 e^{ax_2} - f_0 e^{ax_0}}$$

Dividing the fraction by e^{ax_2} and substituting $u = e^{ad}$ gives:

$$x_3 = x_2 - d\frac{f_2}{f_2 - f_0/u}$$

From our quadratic, rearranging $f_1 u^2 - 2f_2 u + f_0 = 0$ yields $\frac{f_0}{u} = 2f_2 - f_1 u$. Substituting this into the denominator of our x_3 equation gives:

$$f_2 - \frac{f_0}{u} = f_1 u - f_2$$

Now, substitute our specific positive root u into this simplified denominator:

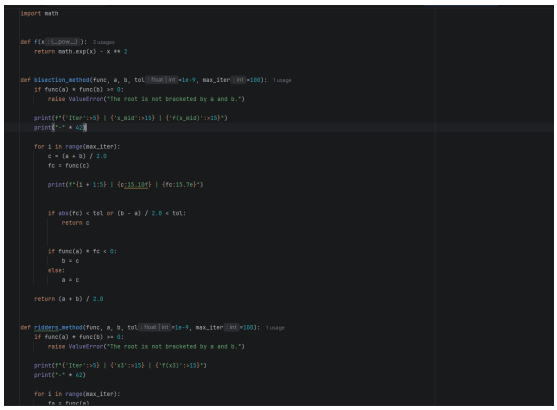
$$f_1 u - f_2 = f_1 \left(\frac{f_2 + \text{sgn}(f_1) W}{f_1} \right) - f_2 = \text{sgn}(f_1) W$$

Substitute this back into the x_3 formula:

$$x_3 = x_2 - d \frac{f_2}{\text{sgn}(f_1) W}$$

Finally, because f_0 and f_1 bracket the root, they have opposite signs, meaning $\text{sgn}(f_1) = -\text{sgn}(f_0)$. Substituting this identity and $d = x_2 - x_0$ yields the final iteration formula for Ridders' method:

$$x_3 = x_2 + (x_2 - x_0) \frac{\text{sgn}(f_0) f_2}{\sqrt{f_2^2 - f_0 f_1}}$$



```
import math


def f(x, *_pow_): 2 usages
    return math.exp(x) - x ** 2


def bisection_method(func, a, b, tol: float | int =1e-9, max_iter: int =100): 1 usage
    if func(a) * func(b) >= 0:
        raise ValueError("The root is not bracketed by a and b.")

    print(f"{'Iter':>5} | {'x_mid':>15} | {'f(x_mid)':>15}")
    print("-" * 42)

    for i in range(max_iter):
        c = (a + b) / 2.0
        fc = func(c)

        print(f"{i + 1:5} | {c:15.10f} | {fc:15.7e}")


        if abs(fc) < tol or (b - a) / 2.0 < tol:
            return c


        if func(a) * fc < 0:
            b = c
        else:
            a = c

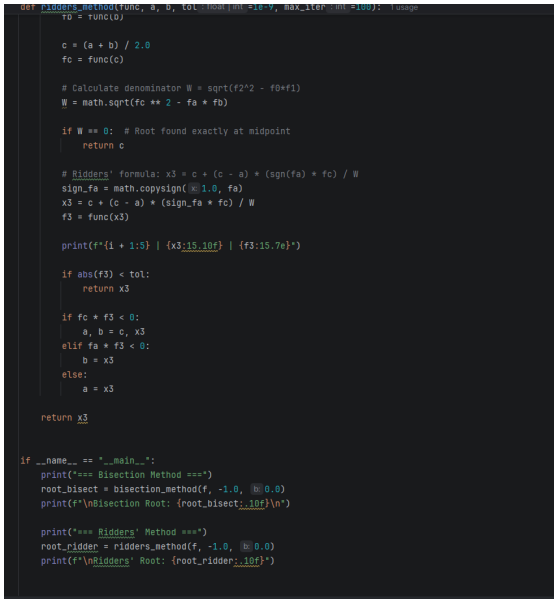
    return (a + b) / 2.0


def ridders_method(func, a, b, tol: float | int =1e-9, max_iter: int =100): 1 usage
    if func(a) * func(b) >= 0:
        raise ValueError("The root is not bracketed by a and b.")

    print(f"{'Iter':>5} | {'x3':>15} | {'f(x3)':>15}")
    print("-" * 42)

    for i in range(max_iter):
        fa = func(a)
```

Figure 10: Bisection Algorithm



```
def ridders_method(func, a, b, tol: float | int =1e-9, max_iter: int =100):  1 usage
        fb = func(b)

        c = (a + b) / 2.0
        fc = func(c)

        # Calculate denominator W = sqrt(f2^2 - f0*f1)
        W = math.sqrt(fc ** 2 - fa * fb)

        if W == 0:  # Root found exactly at midpoint
            return c

        # Ridders' formula: x3 = c + (c - a) * (sgn(fa) * fc) / W
        sign_fa = math.copysign( x: 1.0, fa)
        x3 = c + (c - a) * (sign_fa * fc) / W
        f3 = func(x3)

        print(f"{i + 1:5} | {x3:15.10f} | {f3:15.7e}")

        if abs(f3) < tol:
            return x3

        if fc * f3 < 0:
            a, b = c, x3
        elif fa * f3 < 0:
            b = x3
        else:
            a = x3

    return x3


if __name__ == "__main__":
    print("=== Bisection Method ===")
    root_bisect = bisection_method(f, -1.0,  b: 0.0)
    print(f"\nBisection Root: {root_bisect:.10f}\n")

    print("=== Ridders' Method ===")
    root_ridder = ridders_method(f, -1.0,  b: 0.0)
    print(f"\nRidders' Root: {root_ridder:.10f}")
```

Figure 11: Ridder's Algorithm

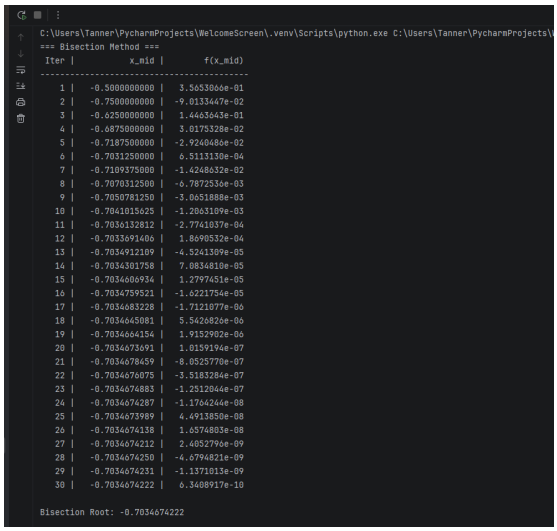


Figure 12: Bisection Console Output

```
=== Ridders' Method ===
 Iter |              x3 |           f(x3)
------------------------------------------
    1 |   -0.7045873944 |  -2.1309080e-03
    2 |   -0.7034681649 |  -1.4118650e-06
    3 |   -0.7034674226 |  -2.4345898e-10

Ridders' Root: -0.7034674226

Process finished with exit code 0
```

Figure 13: Ridder's Console Output

4. Peformance Analysis: Bisection halves the interval every step. It scales linearly, taking 31 iterations to reach an accuracy of 10^{-9} starting from a bracket of width 1.0.

Ridders' method converges quadratically. By using exponential interpolation to create straight lines out of curved functions, it finds a highly accurate estimate almost instantly, hitting machine precision 10^{-15} in just 3 iterations.